Microelectronics Journal 173 (2026) 107137

Contents lists available at ScienceDirect

Microelectronics Journal

journal homepage: www.elsevier.com/locate/mejo

NLFSR PUF: A lightweight and reliable strong PUF with modeling-attack resistance☆

Zhengfeng Huang [a], Xingchen Du [a], Chenxing Su [a], Jingchang Bian [b,*]

[a] School of Microelectronics, Hefei University of Technology, Hefei 230601, China
[b] School of Integrated Circuits, Anhui Polytechnic University, Wuhu 241000, China

ARTICLE INFO

Keywords:
Hardware security
Physical Unclonable Functions
Machine learning
Nonlinear feedback shift register

ABSTRACT

The rapid growth of IoT devices raises critical security challenges, as limited resources hinder conventional cryptographic use. Physical Unclonable Functions (PUFs) offer a hardware-based solution by exploiting manufacturing variations for authentication and key generation. However, strong PUFs remain vulnerable to machine-learning attacks. To address this, we propose a lightweight obfuscation framework based on a nonlinear feedback shift register (NLFSR). By introducing strong nonlinearity and internal state into the challenge–response mapping, the framework disrupts linear delay models used in modeling attacks. FPGA experiments demonstrate that the proposed design reduces the prediction accuracy of advanced learning models to about 50%, equivalent to random guessing. Compared with existing lightweight PUFs, the NLFSR PUF lowers area overhead by nearly 33%, while achieving 96.47% reliability and 49.99% uniqueness. Reliability ranges from 91.66% to 96.47% across −10 °C to 60 °C, with 93.40% at 0.90 V, 96.47% at 1.00 V, and 88.79% at 1.10 V. Its randomness also passes the NIST SP800-22 test.

1. Introduction

The rapid proliferation of Internet of Things (IoT) devices has transformed industries ranging from healthcare and transportation to smart homes and industrial automation, significantly enhancing connectivity, automation, and convenience [1]. However, the ubiquitous deployment of IoT devices introduces critical security concerns, as many of these devices operate in resource-constrained environments with limited computational and storage capacities, making conventional cryptographic approaches impractical [2]. Consequently, ensuring secure device authentication, data integrity, and protection against counterfeiting and cloning attacks have become increasingly challenging. In this context, PUFs have emerged as a promising hardware-based security solution.

PUFs have emerged as a compelling hardware root-of-trust for lightweight security, leveraging inherent physical variations in integrated circuits to generate unique identifiers [3]. Due to their intrinsic unpredictability and unclonability, PUFs offer robust solutions for device authentication, cryptographic key generation, and intellectual property protection, particularly in resource-constrained environments. Depending on the size of their challenge–response space, PUFs are commonly classified as *weak* or *strong* [4]. In strong PUFs, the vast number of challenge–response pairs (CRPs) is essential for authentication, yet their exposure over time renders the PUF vulnerable to machine learning (ML) modeling attacks. Rührmair et al. demonstrated that logistic regression can predict the responses of Arbiter PUF (APUF), XOR APUF, Feed-Forward APUF and Lightweight Secure PUF with accuracies exceeding 90% given only a modest training set [5,6]. Subsequent work confirmed that evolutionary strategies and neural networks constitute similarly potent modeling threats [7].

To thwart such attacks, many recent designs embed an Arbiter-based core within auxiliary structures that obfuscate the challenge, response or internal delay paths. Linear feedback shift registers (LFSRs) have been widely adopted for that purpose. For example, the CRC-PUF interleaves an LFSR with the APUF delay chain to confuse the ML model [8]. Nevertheless, once an attacker recovers the feedback polynomial and initial seed, the linear nature of the LFSR permits algebraic inversion. Follow-up schemes such as SR-PUF [9], DCH-PUF and DCT-PUF [10,11] therefore refresh the feedback taps dynamically, but at the cost of considerable hardware overhead and degraded reliability. FLAM-PUF focuses on minimizing hardware cost by using simple components like the APUF, LFSR, and basic logic gates. However, its linear feedback mechanism leads to noise accumulation, which compromises long-term reliability [12]. BF-PUF, on the other hand, enhances resistance to modeling attacks by leveraging the nonlinearity of Bent functions and Feistel networks. This comes at the

☆ This work was supported in part by the National Natural Science Foundation of China under Grant nos. 62274052, 62504003.
* Corresponding author.
E-mail addresses: huangzhengfeng@139.com (Z. Huang), xingcdu@163.com (X. Du), 15053791376@163.com (C. Su), jingchangbian@126.com (J. Bian).

https://doi.org/10.1016/j.mejo.2026.107137
Received 1 September 2025; Received in revised form 29 December 2025; Accepted 27 February 2026
Available online 28 February 2026

Nomenclature

ω	Delay weight vector
$\boldsymbol{\Phi}$	Challenge feature vector
c	n-bit challenge
$\Delta\tau$	Total delay difference of n-stage APUF
h_i	(i + 1)th tap coefficient
L	NLFSR length (number of flip-flops)
q	Number of operation cycles of NLFSR
R	1-bit final response (0 or 1)
x_i	(i + 1)th flip-flop value in NLFSR
z_i	(i + 1)th input of bent function

cost of significantly higher hardware complexity, making it less suitable for resource-constrained environments [13]. More fundamentally, the linear recurrence of any LFSR leaves it susceptible to algebraic and correlation attacks, a fact well-documented in the stream-cipher literature [14–16].

Motivated by these shortcomings, this work proposes a lightweight obfuscation framework that couples strong PUF primitives with an NLFSR. The NLFSR drastically enlarges the state transition space while preserving a compact footprint, thereby impeding algebraic recovery and ML modeling. Our main contributions are summarized below.

- We propose a generic obfuscation scheme based on NLFSR, which can be integrated with existing strong PUFs. It injects strong nonlinearity and internal state into the challenge–response mapping, breaks the linear delay model exploited by ML so that accurate modeling requires impractically many CRPs.
- We derive the noise propagation formulas for noise propagation in both LFSR and NLFSR, and prove that the NLFSR's output noise decays exponentially to a bounded floor, thus preserving response reliability.
- We implement a 64-stage NLFSR PUF that attains 96.47% reliability without CRP filtering while consuming fewer hardware resources than the state-of-the-art alternatives. Comprehensive evaluations confirm the uniqueness and randomness of the generated responses, making the design well suited to resource-limited IoT nodes. FPGA experiments on a Xilinx Artix-7 demonstrate that the proposed NLFSR PUF significantly reduces the prediction accuracy of logistic regression (LR), Covariance Matrix Adaptation Evolution Strategy (CMA-ES), Artificial Neural Networks (ANN), Deep Neural Networks (DNN) and Support Vector Machine (SVM). The accuracy is reduced to approximately that of a random guess (≈50%).

The remainder of this article is organized as follows. Section 2 reviews the required background. Section 3 details the proposed NLFSR PUF architecture. Section 4 analyzes noise propagation and outlines an adaptive response-selection strategy. FPGA implementation results are presented in Section 5. Finally, Section 6 concludes this article.

2. Background

2.1. Arbiter PUF and mathematical model

Since the introduction of the Arbiter PUF, it has quickly become the most popular strong PUF circuit. As shown in Fig. 1, this structure consists of cascaded n crossbar switches, each composed of two multiplexers. When the challenge signal (c_i) is 0, the upper and lower signals cross the crossbar switch through a parallel path, and when c_i is 1, they cross the crossing path. In an ideal situation, both paths would experience the same delay when given the same challenge signal. However, in reality, the delay will differ slightly due to manufacturing variations. At the end of the chain, an arbiter is used to compare the delay difference between the top and bottom paths. The arbiter is typically constructed with a latch or D flip-flop.

The additive linear delay model is a standard approach to describe APUF functionality. It explains the time delay difference between the top and bottom paths [17–19]. The input challenge of the APUF is an n-bit vector defined as $\boldsymbol{c} = (c_0, c_1, \ldots, c_{n-1})$. At each stage i, the propagation path is determined by the corresponding challenge bit c_i. The delay characteristics of each stage are illustrated in Fig. 2. For the ith switch, the delay information of its four internal paths is denoted $a_{i,0}$, $a_{i,1}$, $a_{i,2}$, and $a_{i,3}$, respectively. When $c_i = 0$ and $c_i = 1$, the delay difference between two symmetrical paths are given by

$$\tau_i^0 = \alpha_{i,0} - \alpha_{i,3} \quad \text{and} \quad \tau_i^1 = \alpha_{i,1} - \alpha_{i,2}, \tag{1}$$

respectively. For a specific input c_i, the sum of the delay difference $\Delta\tau_i$ to the stage i can be positive or negative. So we can obtain

$$\Delta\tau_i = (1 - 2c_i) \cdot \Delta\tau_{i-1} + (1 - c_i) \cdot \tau_i^0 + c_i \cdot \tau_i^1 \tag{2}$$

for $i > 0$. The delay vector is $\boldsymbol{\omega} = (\omega_0, \omega_1, \ldots, \omega_i, \ldots, \omega_n)$ with

$$\begin{aligned} \omega_0 &= \frac{\tau_0^0 - \tau_0^1}{2}, \\ \omega_i &= \frac{\tau_{i-1}^0 + \tau_{i-1}^1 + \tau_i^0 - \tau_i^1}{2}, \\ \omega_n &= \frac{\tau_{n-1}^0 + \tau_{n-1}^1}{2}. \end{aligned} \tag{3}$$

And the feature vector $\boldsymbol{\Phi} = (\phi_0, \phi_1, \ldots, \phi_l, \ldots \phi_{n-1}, 1)^T$ is determined by challenge c_i :

$$\phi_l = \prod_{i=l}^{n-1} (1 - 2c_i) \tag{4}$$

Accordingly, we can express the final delay difference $\Delta\tau_{n-1}$ as

$$\begin{aligned} \Delta\tau_{n-1} &= \omega_0 \prod_{i=0}^{n-1} (1 - 2c_i) + \omega_1 \prod_{i=1}^{n-1} (1 - 2c_i) + \cdots + \omega_n \\ &= \boldsymbol{\omega} \cdot \boldsymbol{\Phi} \end{aligned} \tag{5}$$

The APUF response at the arbiter output can be further expressed as

$$R = \mathrm{H}(\boldsymbol{\omega} \cdot \boldsymbol{\Phi}) = \mathrm{H}(\Delta\tau_{n-1}) \tag{6}$$

where $\mathrm{H}(\cdot)$ denotes the Heaviside step function, R is -1 if $\Delta\tau_{n-1} \le 0$, else R is 1.

2.2. The concept of LFSR and NLFSR

LFSR is among the most widely used tools for generating and managing pseudorandom sequences. Certain lightweight secure-enhanced strong PUF designs incorporate LFSR as a method to obfuscate both challenges and responses [20,21]. Fig. 3 shows an n-stage Fibonacci LFSR consisting of n flip-flops with connected XOR gates. The transfer function of this LFSR can be described using a matrix of size $n \times n$, denoted as **Fb**, which is known as the characteristic matrix. The characteristic matrix takes the following form:

$$\mathbf{Fb} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 & 0 \\ 0 & 0 & 1 & \cdots & 0 & 0 \\ 0 & 0 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & \cdots & 1 \\ h_0 & h_1 & h_2 & \cdots & h_{n-2} & h_{n-1} \end{bmatrix} \tag{7}$$

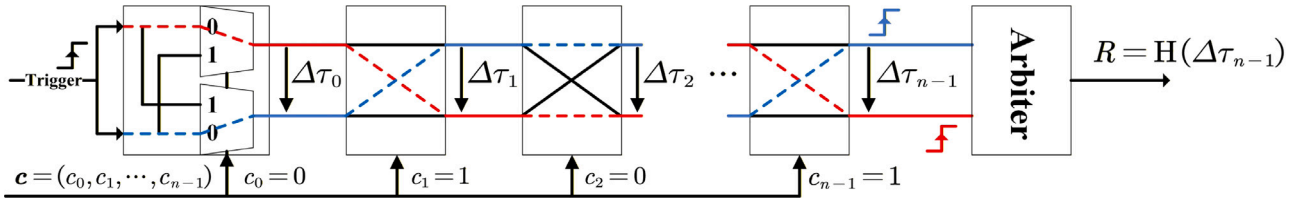

Fig. 1. Structure of a traditional APUF.

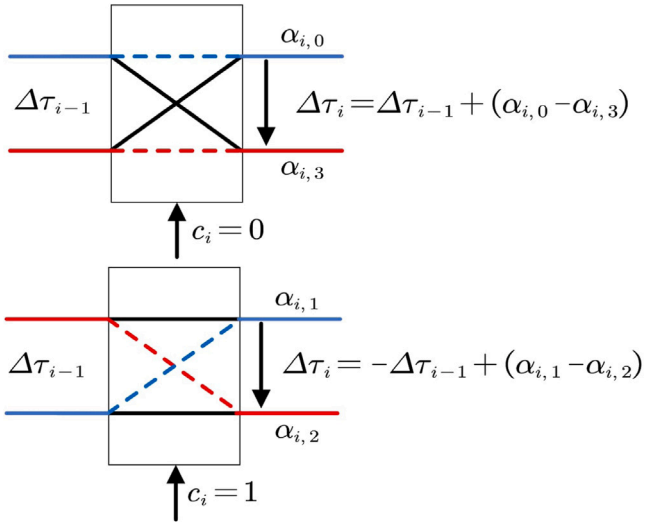

Fig. 2. Cascaded delay path modeling of a traditional APUF. Depending on the challenge bit $c_i = 0$ or $c_i = 1$, the signal propagates through different paths in the delay stage, contributing to the accumulated delay difference.

where $h_i \in \{0, 1\}, i = 0, 1, \ldots, n-1$. The characteristic polynomial of a LFSR is obtained as the determinant of the matrix $\mathbf{Fb} - \mathbf{I}x$, i.e.,

$$
\begin{aligned}
f(x) &= \det(\mathbf{Fb} - \mathbf{I}x) \\
&= \begin{vmatrix}
x & 1 & 0 & \cdots & 0 & 0 \\
0 & x & 1 & \cdots & 0 & 0 \\
0 & 0 & x & \cdots & 0 & 0 \\
\vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\
0 & 0 & 0 & \cdots & x & 1 \\
h_0 & h_1 & h_2 & \cdots & h_{n-2} & h_{n-1} + x
\end{vmatrix} \\
&= x^n + h_{n-1}x^{n-1} + \cdots + h_1 x + h_0
\end{aligned}
\tag{8}
$$

where the term $h_i x^i$ refers to the $(i+1)$th flip-flop of the register [22]. If $h_i = 1$, then a feedback tap is taken from this flip-flop. Fig. 4 illustrates the Galois LFSR, characterized by its unique structure where the output from the rightmost stage is fed back into specific stages, as defined by the following characteristic matrix:

$$
\mathbf{Gl} = \begin{bmatrix}
h_0 & 1 & 0 & 0 & \cdots & 0 \\
h_1 & 0 & 1 & 0 & \cdots & 0 \\
h_2 & 0 & 0 & 1 & \cdots & 0 \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
h_{n-2} & 0 & 0 & 0 & \cdots & 1 \\
h_{n-1} & 0 & 0 & 0 & \cdots & 0
\end{bmatrix}.
\tag{9}
$$

It is straightforward to verify that the corresponding characteristic polynomial is $g(x) = \det(\mathbf{Gl} - \mathbf{I}x) = h_{n-1}x^n + h_{n-2}x^{n-1} + \cdots + h_0 x + 1$, where the coefficients h_i are determined by the structure of the characteristic matrix.

Although efficient and well-studied, the linear structure of the LFSR makes it vulnerable to algebraic and correlation attacks in stream cipher applications. To overcome these weaknesses, recent designs

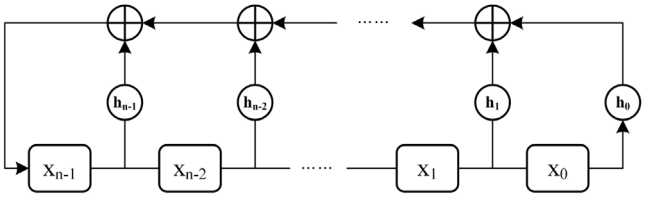

Fig. 3. Fibonacci LFSR.

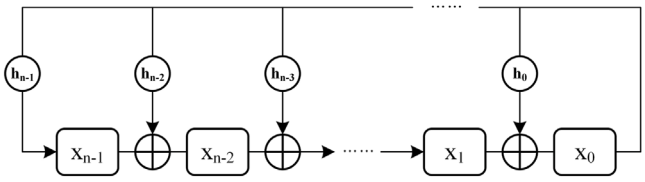

Fig. 4. Galois LFSR.

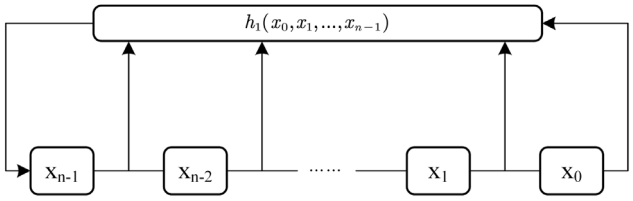

Fig. 5. The nonlinear feedback shift register.

increasingly adopt the NLFSR for improved resistance to such attacks [14–16]. As illustrated in Fig. 5, an n-stage NLFSR uses a Boolean feedback function $h_1 : \mathbb{F}_2^n \to \mathbb{F}_2$. The n binary storage cells hold values $x_0, x_1, \ldots, x_{n-1}$, forming the state $\begin{bmatrix} x_0 & x_1 & \cdots & x_{n-1} \end{bmatrix}^T$. At each clock cycle, the state is shifted left, with the feedback value appended: $\begin{bmatrix} x_1 & \cdots & x_{n-1} & h_1(x_0, x_1, \ldots, x_{n-1}) \end{bmatrix}^T$. An output sequence $\underline{a} = (a_0, a_1, \cdots)$ generated by the NLFSR is a binary sequence that satisfies the following recurrence relation:

$$
a_{i+n} = h_1(a_i, a_{i+1}, \ldots, a_{i+n-1}), \quad i \geq 0. \tag{10}
$$

Let

$$
h(x_0, x_1, \ldots, x_n) = h_1(x_0, x_1, \ldots, x_{n-1}) \oplus x_n \in \mathcal{B}_{n+1}. \tag{11}
$$

Then Eq. (10) can be rewritten as

$$
h(a_i, a_{i+1}, \ldots, a_{i+n}) = 0, \quad i \geq 0. \tag{12}
$$

That is, $h(\underline{a}) = \underline{0}$. We call h the characteristic function of the NLFSR and denote the NLFSR by NLFSR(h). Denote by $S(h)$ the set of all 2^n output sequences of NLFSR(h), defined as

$$
S(h) = \left\{ \underline{a} \in \mathbb{F}_2^{\infty} \mid h(\underline{a}) = \underline{0} \right\}. \tag{13}
$$

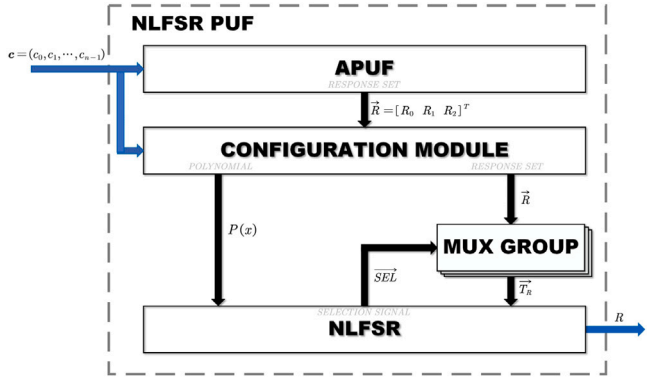

Fig. 6. Overall framework of the proposed NLFSR PUF architecture.

3. Proposed NLFSR PUF

3.1. Overview of proposed PUF

Fig. 6 illustrates the structural overview of the proposed NLFSR PUF. This architecture comprises the following four modules:

- **APUF Module**: A conventional Arbiter PUF circuit comprising an n-stage symmetrical switching structure.
- **Configuration Module**: Responsible for generating polynomial functions $P(x)$ utilized by the NLFSR.
- **MUX Group**: Employs selection signals ($\overrightarrow{SEL}$) generated by the NLFSR module to dynamically transform the intermediate response vector ($\vec{R}$) into a round-dependent vector ($\overrightarrow{T_R}$).
- **NLFSR Module**: Generates the final response R by applying the polynomial functions $P(x)$ along with the intermediate response vector $\overrightarrow{T_R}$.

The operation of the proposed NLFSR PUF framework can be summarized through the following steps:

Step 1: **Initial Response Generation.** For a given n-bit challenge vector c, the APUF module initially produces a raw response sequence. After ensuring that the response has reached a stable state, three specific response bits are selected in accordance with the procedure described in Section 4.2. In this work, the selected bits correspond to the 63rd, 31st, and 15th positions within the stabilized response sequence.

Step 2: **Polynomial Generation.** The configuration module generates the polynomial function $P(x)$ for the NLFSR based on the initial challenge c and the three-bit response obtained from Step 1.

Step 3: **NLFSR Execution and Final Response Generation.** The NLFSR is initialized using the generated polynomial function $P(x)$ and operates for a specified number of rounds. Simultaneously, the MUX group leverages the selection signals ($\overrightarrow{SEL}$) output by the NLFSR to produce the intermediate response vector $\overrightarrow{T_R}$. After the NLFSR process stabilizes, the system outputs the final response R.

3.2. NLFSR structure and functionality

Fig. 7 illustrates the architecture of the proposed NLFSR. The NLFSR module comprises a circular array of memory elements integrated with a nonlinear feedback network, which implements combinational logic functions to achieve the desired cryptographic properties. The nonlinear combinational feedback functions within the NLFSR are formally defined as:

$$f\left(z_n, \ldots, z_1, z_0\right) = b\left(z_n, \ldots, z_1\right) + z_k z_0, \quad k \in [1, n], \tag{14}$$

where each variable z_i represents an input bit sourced from one of the flip-flops within the NLFSR. The function $b(z_n, \ldots, z_1)$ denotes an n-variable bent function [23,24]. In this study, the parameter n is set to 4.

A bent function represents the highest achievable nonlinearity among all Boolean functions with a given number of inputs. Specifically, for an n-variable bent function, the output attains the value zero exactly $2^{n-1} + 2^{n/2-1}$ times. In contrast, the modified function $b(z_n, \ldots, z_1) + z_k$ evaluates to zero exactly $2^{n-1} - 2^{n/2-1}$ times. As established by Seberry and Zhang [25], the function $f(z_n, \ldots, z_1, z_0)$ is balanced provided that the two aforementioned conditions are satisfied. Consequently, the constructed function $f(z_n, \ldots, z_1, z_0)$ is balanced; that is, for uniformly random input vectors, the output values 0 and 1 occur with equal probability, each being exactly 0.5. Furthermore, function $f(z)$ demonstrates strong nonlinearity, satisfying $N_g \geq 2^n - 2^{n/2}$.

The choice of NLFSR feedback functions $f(z)$ is systematically made to ensure pairwise distinct bent functions without mutual complementarity. The selection is further guided by evaluating the Algebraic Normal Form (ANF) representations of these functions. For instance, considering the set of 3584 valid 5-input functions conforming to Eq. (14), their ANF representations might yield expressions such as:

$$\begin{aligned} f_1(z_4, z_3, z_2, z_1, z_0) &= z_1 + z_4 z_0 + z_3 z_2 + z_1 z_0, \\ f_2(z_4, z_3, z_2, z_1, z_0) &= z_4 + z_1 + z_4 z_2 + z_3 z_2 + z_3 z_1 \\ &\quad + z_3 z_0 + z_2 z_1 + z_2 z_0 + z_1 z_0. \end{aligned} \tag{15}$$

In scenarios where multiple candidate functions have the same number of input bits, preference is given to functions containing the minimum number of logical AND operations, facilitating area-efficient hardware implementation.

The generation rule for the round-dependent vector $\overrightarrow{T_R}$ is defined as follows:

$$\begin{aligned} T_{R0} &= \mathrm{NLFSR}_1[11] ? R_0 : R_1, \\ T_{R1} &= \mathrm{NLFSR}_2[11] ? R_1 : R_2. \end{aligned} \tag{16}$$

The initial seed for the NLFSR is derived by transmitting the first 32 bits of the original challenge vector from the configuration module. The operational cycle count of the NLFSR is dynamically controlled by the intermediate response vector $\vec{R}$. For cryptographic robustness, the NLFSR must perform a minimum of q operational cycles, where q exceeds the size L of the NLFSR module. The cycle length is explicitly computed as:

$$q = 4R_0 + 2R_1 + R_2 + L. \tag{17}$$

4. Mathematical analysis

4.1. Noise propagation in LFSR and NLFSR

LFSRs and NLFSRs are fundamental hardware primitives for pseudo-random sequence generation and lightweight cryptography. To quantify how additive noise propagates inside these two structures, we first formalize their state–space representations.

For an n-stage Fibonacci LFSR, the one-clock update matrix is

$$\mathbf{Fb} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 & 0 \\ 0 & 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & 0 \\ 0 & 0 & 0 & \cdots & 0 & 1 \\ h_0 & h_1 & h_2 & \cdots & h_{n-2} & h_{n-1} \end{bmatrix}, \quad h_i \in \{0, 1\}. \tag{18}$$

whereas an n-stage Galois LFSR is governed by

$$\mathbf{Gl} = \begin{bmatrix} h_0 & 1 & 0 & 0 & \cdots & 0 \\ h_1 & 0 & 1 & 0 & \cdots & 0 \\ h_2 & 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ h_{n-2} & 0 & 0 & 0 & \cdots & 1 \\ h_{n-1} & 0 & 0 & 0 & \cdots & 0 \end{bmatrix}. \tag{19}$$

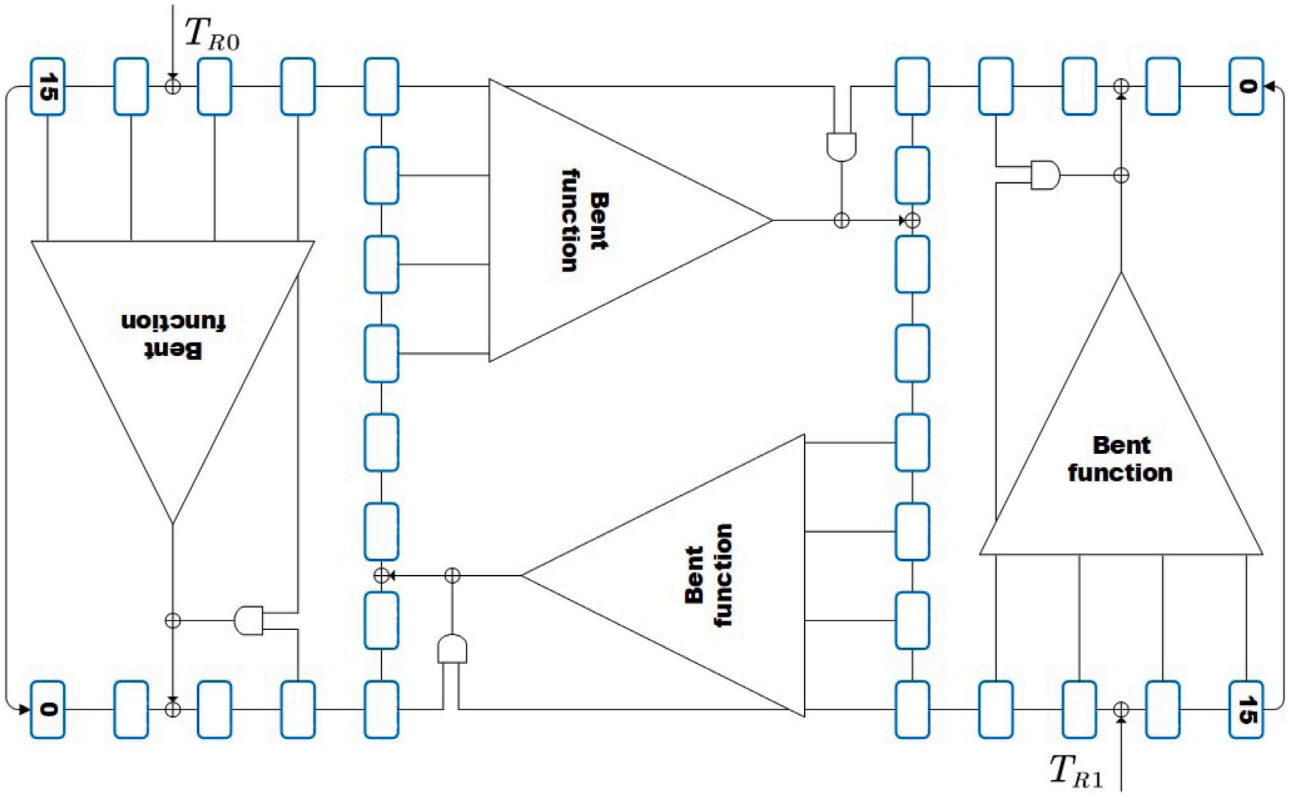

Fig. 7. Structure of the NLFSR module.

Embedding either 0–1 matrix in $\mathbb{R}^{n\times n}$ and superposing independent Gaussian disturbances $N_k \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,\sigma^2 I_n)$ yields the discrete stochastic system

$$S_{k+1} = A\,S_k + N_k, \quad \begin{cases} A \in \{\mathbf{Fb}, \mathbf{Gl}\}, \\ S_0 \perp N_k, \\ \mathbb{E}[S_0] = 0 \end{cases} \tag{20}$$

Expanding the recursion gives

$$S_k = A^k S_0 + \sum_{i=0}^{k-1} A^{k-1-i} N_i, \tag{21}$$

hence the state covariance satisfies

$$\Sigma_k = \sigma^2 \sum_{i=0}^{k-1} A^i (A^i)^\top. \tag{22}$$

If the spectral radius $\rho(A) < 1$ the Neumann series in (22) converges and its limit Σ_∞ solves the discrete Lyapunov equation $A\Sigma_\infty A^\top + \sigma^2 I_n = \Sigma_\infty$; for $\rho(A) = 1$ the covariance grows linearly in k; and for $\rho(A) > 1$ it diverges exponentially as $\rho(A)^{2k}$. Hence the feedback-tap vector $(h_0, \ldots, h_{n-1})$ fully determines LFSR noise behavior.

For an NLFSR, define the Boolean feedback $h_1 : \mathbb{F}_2^n \to \mathbb{F}_2$ and the update function

$$g\left(\begin{bmatrix} x_0 \\ \vdots \\ x_{n-1} \end{bmatrix}\right) = \begin{bmatrix} x_1 \\ \vdots \\ x_{n-1} \\ h_1(x_0, \ldots, x_{n-1}) \end{bmatrix}. \tag{23}$$

After real embedding and the same Gaussian perturbation, the state equation becomes

$$S_{k+1} = g(S_k) + N_k. \tag{24}$$

Its Jacobian at S_k is

$$F_k = \begin{bmatrix} 0 & 1 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \\ \partial_0 h_1(S_k) & \partial_1 h_1(S_k) & \cdots & \partial_{n-1} h_1(S_k) \end{bmatrix} \tag{25}$$
$$\partial_i h_1(S_k) \in \{0, 1\}.$$

Assume there exists $0 < \alpha < 1$ such that $\|F_k\|_2 \le \alpha$ for all k. Here, α represents a noise factor, which can be interpreted as the reliability of the system. The value of α corresponds to the degree of noise propagation, where α is closer to 0 when the system is highly reliable and α approaches 1 as noise propagation increases. Hence, α takes values within the interval (0, 1), and it characterizes the system's noise behavior. The derivative $\partial_i h_1(S_k)$ is not a classical derivative but should be interpreted as a discrete, Lipschitz-like constant that characterizes the behavior of the Boolean function in the real domain. This discrete approximation reflects how noise propagates in a step-wise, non-continuous manner through the Boolean function. Then the covariance recursion $\Sigma_{k+1} = F_k \Sigma_k F_k^\top + \sigma^2 I_n$ implies

$$\begin{aligned} &\Sigma_{k+1} \preceq \alpha^2 \Sigma_k + \sigma^2 I_n, \\ &\quad \Longrightarrow \ \Sigma_k \preceq \frac{1-\alpha^{2k}}{1-\alpha^2}\,\sigma^2 I_n, \\ &\Sigma_\infty = \frac{\sigma^2}{1-\alpha^2}\, I_n. \end{aligned} \tag{26}$$

Eq. (26) shows that, under the Jacobian-contraction condition, NLFSR noise decays exponentially to a finite bound.

4.2. Strategy for response selection

Although APUFs are designed to generate responses based on inherent circuit delay randomness, certain correlations may persist among these responses, significantly impacting their security and reliability.

Thus, we provide a comprehensive derivation of the correlation between the final APUF response and the intermediate response at bit position i.

When the trigger signal propagates through stage i, we define delay differences as follows:

$$\tau_i^0 = \alpha_{i,0} - \alpha_{i,3} \quad (c_i = 0), \quad \tau_i^1 = \alpha_{i,1} - \alpha_{i,2} \quad (c_i = 1). \tag{27}$$

Setting $\sigma_1^2 = 2\sigma^2$, we assume τ_i^0 and τ_i^1 are independently and identically distributed normal random variables, i.e., $\tau_i^0, \tau_i^1 \sim \mathcal{N}(0, \sigma_1^2)$ [26, 27]. We define the cumulative total delay up to the ith stage as X_1:

$$X_1 = \sum_{k=0}^{i-1} (1 - 2c_k)\tau_k^{(c_k)}. \tag{28}$$

Given that each delay difference $\tau_i^{(c_i)}$ is independently distributed, it follows that $X_1 \sim \mathcal{N}(0, i\sigma_1^2)$.

Similarly, we denote the delays at stage i before and after flipping as X_2 and X_2', respectively. The cumulative delay for the remaining $(n-1-i)$ stages is denoted as X_3. Consequently, we have:

$$X_2,\ X_2' \sim \mathcal{N}(0, \sigma_1^2), \quad X_3 \sim \mathcal{N}(0, (n-1-i)\sigma_1^2). \tag{29}$$

We evaluate the joint probability: $P(X_1 + X_2 > 0, -X_1 + X_2' > 0) \cdot P(X_1 + X_2 + X_3 > 0, -X_1 + X_2' + X_3 > 0)$. To simplify analysis, we define intermediate linear combinations:

$$\begin{aligned} Y_1 &= X_1 + X_2 + X_3, & Y_2 &= -X_1 + X_2' + X_3, \\ Y_3 &= X_1 + X_2, & Y_4 &= -X_1 + X_2'. \end{aligned} \tag{30}$$

Leveraging independence, we calculate their variances and covariances as:

$$\begin{aligned} &\mathrm{Var}(Y_1) = n\sigma_1^2, && \mathrm{Var}(Y_2) = n\sigma_1^2, \\ &\mathrm{Var}(Y_3) = (i+1)\sigma_1^2, && \mathrm{Var}(Y_4) = (i+1)\sigma_1^2, \\ &\mathrm{Cov}(Y_1, Y_2) = (n-1-2i)\sigma_1^2, && \mathrm{Cov}(Y_3, Y_4) = i\sigma_1^2. \end{aligned} \tag{31}$$

To facilitate standardization, we introduce normalized variables:

$$\begin{aligned} Z_1 &= \frac{Y_1}{\sqrt{n\sigma_1^2}}, & Z_2 &= \frac{Y_2}{\sqrt{n\sigma_1^2}}, \\ Z_3 &= \frac{Y_3}{\sqrt{(i+1)\sigma_1^2}}, & Z_4 &= \frac{Y_4}{\sqrt{(i+1)\sigma_1^2}}. \end{aligned} \tag{32}$$

The vector (Z_1, Z_2) and (Z_3, Z_4) then follow standard bivariate normal distributions with respective correlation coefficients:

$$\begin{aligned} \rho_1 &= \frac{\mathrm{Cov}(Y_1, Y_2)}{\sqrt{\mathrm{Var}(Y_1)\mathrm{Var}(Y_2)}} = \frac{n-1-2i}{n}, \\ \rho_2 &= \frac{\mathrm{Cov}(Y_3, Y_4)}{\sqrt{\mathrm{Var}(Y_3)\mathrm{Var}(Y_4)}} = \frac{i}{i+1}. \end{aligned} \tag{33}$$

Consequently, the joint probability simplifies to evaluating: $P(Z_1 > 0, Z_2 > 0) \cdot P(Z_3 > 0, Z_4 > 0)$. These probabilities are represented as integrals of the bivariate normal distribution over the first quadrant:

$$\begin{aligned} P(Z_1 > 0, Z_2 > 0) &= \frac{1}{2\pi\sqrt{1-\rho_1^2}} \\ &\iint_{(0,\infty)^2} \exp\left(-\frac{z_1^2 - 2\rho_1 z_1 z_2 + z_2^2}{2(1-\rho_1^2)}\right) dz_1\, dz_2, \\ P(Z_3 > 0, Z_4 > 0) &= \frac{1}{2\pi\sqrt{1-\rho_2^2}} \\ &\iint_{(0,\infty)^2} \exp\left(-\frac{z_3^2 - 2\rho_2 z_3 z_4 + z_4^2}{2(1-\rho_2^2)}\right) dz_3\, dz_4. \end{aligned} \tag{34}$$

For small correlation values (e.g., large n), we can approximate these probabilities as follows:

$$\begin{aligned} &P(Z_1 > 0, Z_2 > 0) \cdot P(Z_3 > 0, Z_4 > 0) \\ &\approx \left(\frac{1}{2} + \frac{1}{\pi}\arcsin(\rho_1)\right)\left(\frac{1}{2} + \frac{1}{\pi}\arcsin(1)\right) \\ &= \frac{1}{2} + \frac{1}{\pi}\arcsin\left(\frac{n-1-2i}{n}\right). \end{aligned} \tag{35}$$

To identify the condition for the joint probability to approximate 0.5, we set:

$$\frac{1}{2} + \frac{1}{\pi}\arcsin(\rho_1) = 0.5 \Rightarrow \rho_1 = 0 \Rightarrow \frac{n-1-2i}{n} = 0, \tag{36}$$

thus solving for i:

$$i = \frac{n-1}{2}. \tag{37}$$

Consequently, the joint probability is approximately 0.5 when the variance parameter i satisfies this relationship.

5. Measurement results and discussions

To evaluate the performance of the proposed NLFSR PUF, 30 instances were implemented in distinct regions of a single Xilinx Artix-7 XC7A200T FPGA. The hardware design and deployment were conducted using the Vivado 2018.3 design suite. A challenge generator was used to send challenges to the PUF and collect a substantial set of CRPs. An LFSR initialized with a randomly generated fixed seed was used as a pseudo-random number generator to produce the challenge set. The resulting CRPs were transmitted to a host computer via UART interface for subsequent analysis. Experimental data collection and processing were performed on the host system using Python 3.9, running on a Windows 11 platform equipped with 32 GB of RAM and an Intel Core i7-12700H CPU operating at 2.69 GHz.

5.1. Uniqueness

Uniqueness measures the variability between the responses produced by two chips when subjected to the same set of input challenges. It is typically evaluated using the inter-die Hamming Distance (HD). This metric reflects how different the responses from two distinct chips are under identical input conditions. Ideally, the HD between two uncorrelated bitstrings is 0.5. The uniqueness is calculated using the following formula:

$$\text{Uniqueness} = \frac{2}{N_{\text{chip}}\left(N_{\text{chip}} - 1\right)} \sum_{u=1}^{N_{\text{chip}}-1} \sum_{v=u+1}^{N_{\text{chip}}} \frac{HD\left(R_u, R_v\right)}{N_{\text{bit}}} \times 100\% \tag{38}$$

Here, R_u and R_v denote the N_{bit}-length responses generated by chips u and v, respectively, among a total of N_{chip} chips, all evaluated under the same challenge set and standard operating conditions(1.0 V and 25 °C) [28].

Fig. 8 presents the measurement results of the inter-die HD.

The test involves applying 100 K input challenges to 30 instances, and the responses of each instance pair are compared. The observed average inter-die HD is $\mu = 49.99\%$ with a standard deviation of $\sigma = 0.15\%$, demonstrating that the proposed PUF exhibits excellent uniqueness performance.

5.2. Reliability

Reliability serves as a critical metric for assessing the robustness of strong PUF against environmental variations. Ideally, a PUF should consistently produce identical responses when subjected to the same challenges under varying environmental conditions. To evaluate the reliability of the proposed PUF, responses are repeatedly measured under temperature fluctuations ranging from −10 °C to 60 °C, as well as

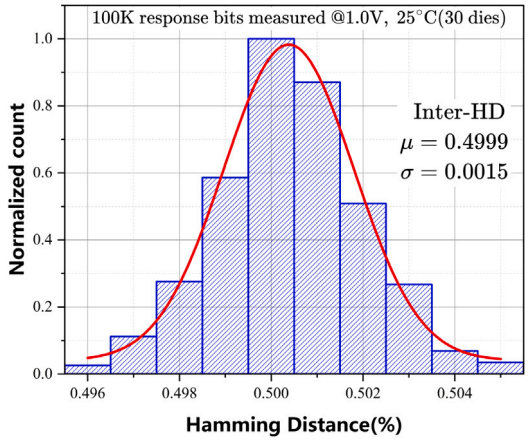

Fig. 8. Histogram of the uniqueness measured as HD_{inter} on all the 30 instance samples.

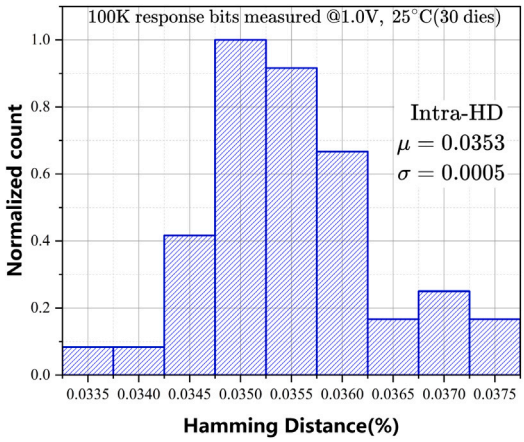

Fig. 9. Histogram illustrating the reliability measured as intra-die HD across all 30 instance samples.

under voltage variations ranging from 0.90 V to 1.10 V. The reliability is quantified by calculating the intra-die HD, as shown in the following equation:

$$\text{Reliability} = 1 - \frac{1}{m}\sum_{j=1}^{m}\frac{HD\left(R_i, R_{i,j}\right)}{n} \times 100\% \tag{39}$$

where R_i represents the n-bit golden responses generated under nominal environmental conditions, and $R_{i,j}$ denotes the n-bit responses obtained by applying the same set of challenges as R_i under varying environmental conditions across m measurements [29]. The reliability of the proposed PUF is evaluated under nominal conditions (a supply voltage of 1.00 V and a temperature of 25 °C). A total of 100 K challenges are tested, with each measurement repeated 10 times on a set of 30 PUFs.

The reliability analysis results are illustrated in Fig. 9. The average intra-die HD is found to be μ = 3.53%, with a standard deviation of σ = 0.05%. Consequently, the overall reliability under nominal conditions is determined to be 96.47 %, demonstrating a highly concentrated reliability distribution among the instance samples. Furthermore, as shown in Fig. 10, the reliability at the lowest temperature (−10 °C) is 91.66 %, and at the highest temperature (60 °C), it is 92.61 %. For the voltage testing, we use the Fusion Digital Power Designer tool to regulate the voltage supplied to the FPGA. The voltage is carefully adjusted within the FPGA's safe operating range (0.90 V to 1.10 V) to ensure reliable performance. The standard voltage is initially set to 1.00 V, with subsequent adjustments made in 0.02 V increments to collect data at each voltage level. As shown in Fig. 11, the reliability at the lowest voltage (0.90 V) is 93.40 %, and at the highest voltage (1.10 V), it is 88.79 %. The reliability increases as the voltage approaches 1.00 V, reaching a peak of 96.47 % at 1.00 V before gradually decreasing again with increasing voltage.

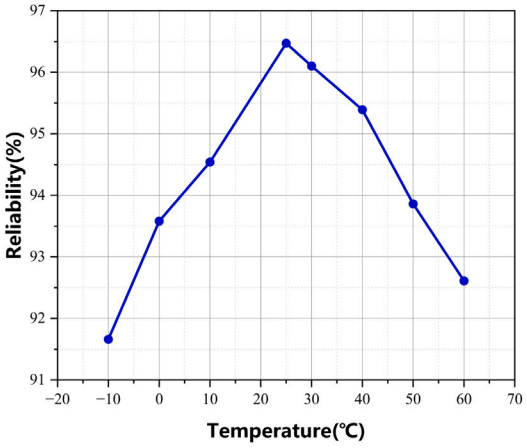

Fig. 10. Reliability of the proposed PUF under varying temperature conditions.

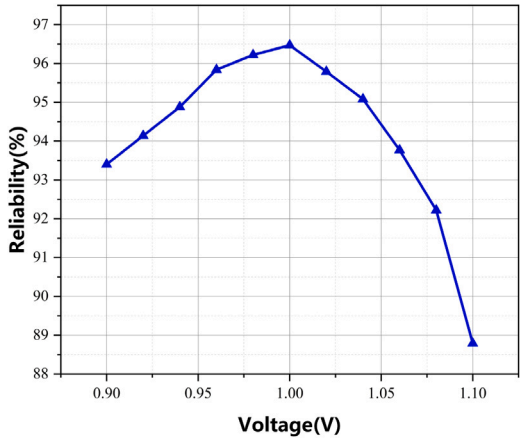

Fig. 11. Reliability of the proposed PUF under varying voltage conditions.

5.3. Randomness

Considering the constraints imposed by limited silicon area in the presence of a large CRP space, the reuse of hardware components may introduce statistical dependencies among CRPs. Randomness is a critical metric for evaluating the unbiased distribution of logical 0 s and 1 s in PUF responses [30]. To assess this property, the NIST SP800-22 statistical test suite is employed [31]. According to the suite, a P-value greater than 0.01 indicates that the data is statistically random, with 99 % confidence. In this work, a 1 M bitstream is generated from the measured instance samples. As shown in Table 1, the bitstream successfully passes all tests in the NIST SP800-22 suite. Meanwhile, the 95 % confidence bound of the autocorrelation function (ACF) is

Table 1
NIST SP800-22 test suite results (1 M response bits).

NIST test items	P-value	Result
Frequency	0.911	Pass
Block frequency	0.911	Pass
Cumulative sums (Forward)	0.739	Pass
Cumulative sums (Reverse)	0.350	Pass
Runs	0.350	Pass
Longest run of ones	0.213	Pass
Rank	0.739	Pass
Discrete Fourier transform (FFT)	0.350	Pass
Non-overlapping template matching	#	Pass
Overlapping template matching	0.534	Pass
Approximate entropy	0.534	Pass
Serial (Forward)	0.350	Pass
Serial (Reverse)	0.350	Pass
Linear complexity	0.350	Pass
Random excursions	#	Pass
Random excursions variant	#	Pass

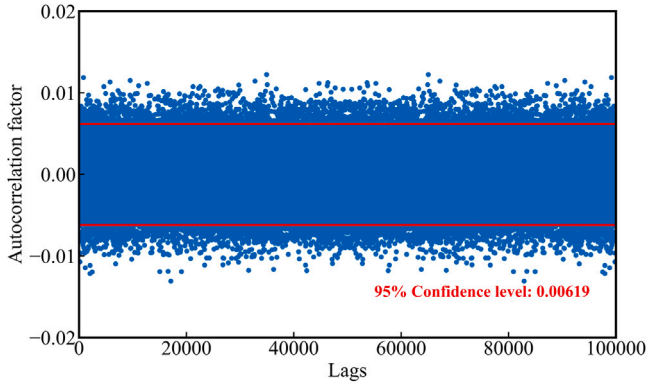

Fig. 12. Autocorrelation results of 100 K consecutive responses.

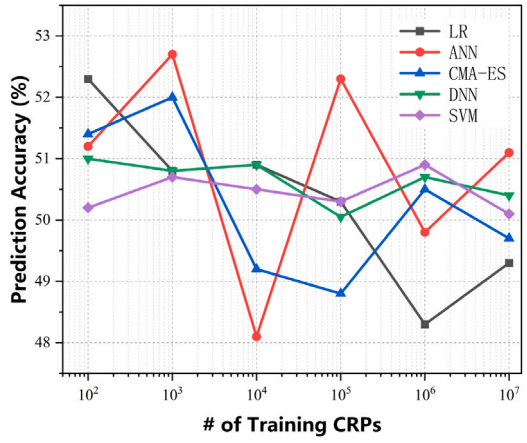

Fig. 13. Prediction accuracies of the ML attacks, including LR, CMA-ES, ANN, DNN, SVM.

±0.006, as shown in Fig. 12. This bound is close to the ideal value of zero, further indicating a high degree of randomness in the generated bitstream.

5.4. Immunity to machine learning attacks

In this section, we evaluate the prediction accuracy of various modeling attacks using well-known machine learning algorithms to assess the resistance of our PUF. A strong PUF can theoretically be regarded as a black-box function $f(\cdot)$, that maps input challenges to corresponding output responses. By acquiring a subset of CRPs, an attacker may apply machine learning techniques to build a model that closely approximates $f(\cdot)$, thus enabling accurate predictions of unseen responses. Such modeling vulnerabilities are particularly prevalent in existing strong PUFs, notably the APUF, whose underlying functions exhibit linear properties that make them vulnerable to ML attacks. Consequently, the ability to resist machine learning-based modeling attacks is a critical performance metric for strong PUFs [17].

Logistic Regression (LR) is a binary classification algorithm that estimates output probabilities using the logistic (sigmoid) function and optimizes its parameters through gradient descent. Due to its ability to effectively predict PUF responses, LR has been widely adopted in modeling attacks against PUFs [32]. The Covariance Matrix Adaptation Evolution Strategy (CMA-ES) is a stochastic optimization method designed for solving complex, non-linear optimization problems in continuous domains. It iteratively updates a covariance matrix to adapt the search direction and scale, thereby improving convergence. In the context of PUF modeling, CMA-ES can capture intricate dependencies between delay parameters, achieving high prediction accuracy for PUF responses [33,34]. Artificial Neural Networks (ANN) is highly flexible machine learning models capable of learning complex, high-dimensional patterns without requiring linear separability or differentiability. When applied to PUFs, ANN has shown strong performance in learning challenge–response mappings, making them a valuable tool for evaluating PUF security [35,36]. Deep Neural Networks (DNN), characterized by their multilayer architectures and nonlinear activation functions, is a powerful black-box approach for addressing complex tasks. In the context of PUF attacks, DNN-based models facilitate high-accuracy black-box attacks, obviating the need for explicit mathematical models of PUFs. Support Vector Machine (SVM) is a binary classification model that uses the Radial Basis Function (RBF) kernel to map data into a higher-dimensional space when classes are not separable in the original space. In PUF attacks, SVM is used to model CRPs, and prediction accuracy is evaluated by comparing the predicted and actual responses. The LR-based attack is conducted with the `pypuf` library [37], adopting its default model configuration. The CMA-ES attack employs the `cma.CMAEvolutionStrategy` module available in Python. Most configuration parameters are kept at their default values. The initial solution vector is set to zero, and the initial standard deviation is $\sigma_0 = 0.5$. The ANN-based attack utilizes a three-layer neural network. The DNN attack is implemented using a two-layer neural network, with the number of neurons in each layer optimized through hyperparameter search within the range of 30 to 300. The SVM attack is performed with a training-to-testing split ratio of 4:1. Fig. 13 illustrates the prediction accuracies of the three employed algorithms. Notably, even after training with up to 10 million CRPs, the prediction accuracies remain close to 50 %, reinforcing the robustness of the system against modeling attacks.

Table 2
GE hardware overhead in 65-nm CMOS technology.

Gate	2-input NAND	2-input AND	2-input XOR	2-to-1 MUX	flip-flop
GE	1	1.5	2.5	2.5	6.25

5.5. Hardware overhead

To fairly evaluate the area cost, we utilize gate equivalent (GE). A GE is a technology-independent unit for measuring circuit area. For CMOS technology, the area of the smallest two-input NAND gate in the library is commonly used as one GE. In this work, the GE values from [8] are employed for area estimation, as illustrated in Table 2.

Table 3 provides a comparison with similar 64-stage designs, evaluated under comparable attack prediction accuracy (approximately 50 %). Each LUT corresponds approximately to 10 to 20 basic logic

Table 3
Hardware cost comparison of NLFSR PUF with other robust PUFs under the same attack prediction accuracy.

PUF type	The number of gates						GE
	2-input NAND	2-input AND	2-input XOR	2-to-1 MUX	flip-flop	Other gates	
NLFSR PUF	–	24	34	130	33	–	652.25
FLAM-PUF [12]	2	62	63	128	65	–	979
8-XOR APUF [4]	–	–	–	1024	8	1	≈2600
Ising-PUF [38]	64	192	–	704	192	≈192	≈3500
(8,8)-iPUF [39]	–	–	–	2064	16	2	≈5300
BF PUF [13]	–	–	–	–	312	405 LUT	≥6000
CT PUF [40]	–	–	–	–	486	731 LUT	≥10,347.5

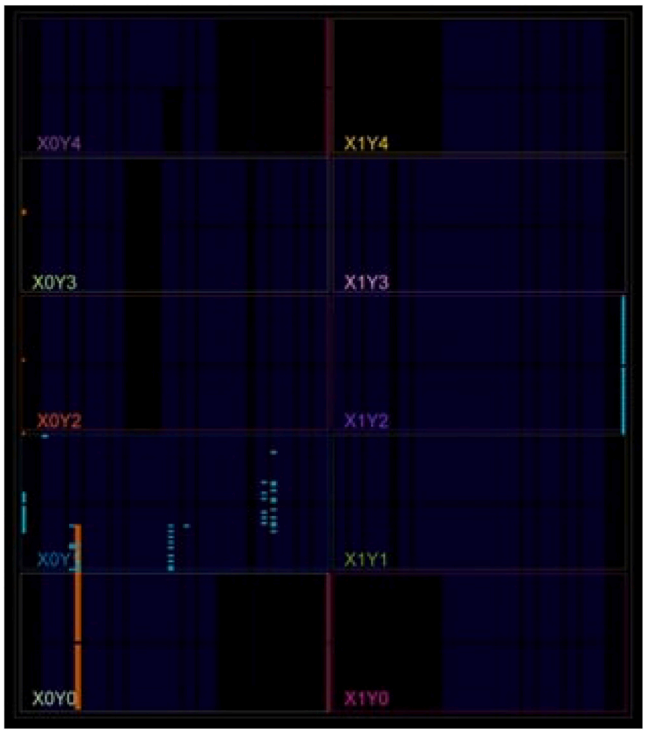

Fig. 14. Hardware layout of the proposed NLFSR PUF.

gates in terms of functionality [12]. Our proposed PUF demonstrates significant superiority in area efficiency over classical robust strong PUF designs, reducing hardware overhead by at least 33 %. Fig. 14 illustrates the hardware implementation layout of the proposed NLFSR PUF. The orange sections represent the APUF module, which is manually constrained through the XDC file to ensure path symmetry, a critical factor for the cryptographic performance of the APUF. In contrast, the cyan sections correspond to the NLFSR module, which does not require strict path symmetry; thus, its placement and routing are automatically optimized by Vivado for implementation efficiency. The design utilizes a total of 188 LUTs.

6. Conclusion

Security and reliability have long been central concerns in the PUF literature. However, improving security often entails increased structural complexity, greater noise, and reduced response stability. This work introduces a lightweight obfuscation framework that integrates strong PUF primitives with an NLFSR, thereby addressing vulnerabilities inherent to conventional LFSR-based designs. The proposed NLFSR PUF exhibits robust resistance to algebraic and machine-learning modeling attacks. FPGA-based evaluations show that the prediction accuracies of LR, CMA-ES, ANN, DNN and SVM attacks are reduced to near-random guessing (approximately 50 %). In addition, the NLFSR-based obfuscation produces exponential decay in noise propagation, effectively stabilizing the output and achieving 96.47 % reliability without any challenge–response pair filtering. Given its combined advantages in security, reliability, and low hardware overhead, the NLFSR PUF architecture is well suited for securing IoT nodes operating under severe resource constraints. Future work will explore further optimizations and broader implementations to extend its practical applicability.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Jingchang Bian reports financial support was provided by National Natural Science Foundation of China. Zhengfeng Huang reports financial support was provided by National Natural Science Foundation of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] P. Mall, R. Amin, A.K. Das, M.T. Leung, K.R. Choo, PUF-based authentication and key agreement protocols for IoT, WSNs, and smart grids: A comprehensive survey, IEEE Internet Things J. 9 (11) (2022) 8205–8228, http://dx.doi.org/10.1109/JIOT.2022.3142084.

[2] X. Yang, L. Shu, Y. Liu, G.P. Hancke, M.A. Ferrag, K. Huang, Physical security and safety of IoT equipment: A survey of recent advances and opportunities, IEEE Trans. Ind. Inform. 18 (7) (2022) 4319–4330, http://dx.doi.org/10.1109/TII.2022.3141408.

[3] C. Herder, M.M. Yu, F. Koushanfar, S. Devadas, Physical unclonable functions and applications: A tutorial, Proc. IEEE 102 (8) (2014) 1126–1141, http://dx.doi.org/10.1109/JPROC.2014.2320516.

[4] G.E. Suh, S. Devadas, Physical unclonable functions for device authentication and secret key generation, in: Proceedings of the 44th Design Automation Conference, DAC 2007, San Diego, CA, USA, June 4-8, 2007, IEEE, 2007, pp. 9–14, http://dx.doi.org/10.1145/1278480.1278484.

[5] U. Rührmair, J. Sölter, PUF modeling attacks: An introduction and overview, in: G.P. Fettweis, W. Nebel (Eds.), Design, Automation & Test in Europe Conference & Exhibition, DATE 2014, Dresden, Germany, March 24-28, 2014, European Design and Automation Association, 2014, pp. 1–6, http://dx.doi.org/10.7873/DATE.2014.361.

[6] U. Rührmair, D.E. Holcomb, PUFs at a glance, in: G.P. Fettweis, W. Nebel (Eds.), Design, Automation & Test in Europe Conference & Exhibition, DATE 2014, Dresden, Germany, March 24-28, 2014, European Design and Automation Association, 2014, pp. 1–6, http://dx.doi.org/10.7873/DATE.2014.360.

[7] L. Kraleva, M. Mahzoun, R. Posteuca, D. Toprakhisar, T. Ashur, I. Verbauwhede, Cryptanalysis of strong physically unclonable functions, 2023, p. 562, IACR Cryptol. ePrint Arch..

[8] E. Dubrova, O. Näslund, B. Degen, A. Gawell, Y. Yu, CRC-PUF: a machine learning attack resistant lightweight PUF construction, in: 2019 IEEE European Symposium on Security and Privacy Workshops, EuroS&P Workshops 2019, Stockholm, Sweden, June 17-19, 2019, IEEE, 2019, pp. 264–271, http://dx.doi.org/10.1109/EUROSPW.2019.00036.

[9] S. Hou, D. Deng, Z. Wang, J. Shi, S. Li, Y. Guo, A dynamically configurable LFSR-based PUF design against machine learning attacks, CCF Trans. High Perform. Comput. 3 (1) (2021) 31–56, http://dx.doi.org/10.1007/S42514-020-00060-7.

[10] Y. Wang, C. Wang, C. Gu, Y. Cui, M. O'Neill, W. Liu, A dynamically configurable PUF and dynamic matching authentication protocol, IEEE Trans. Emerg. Top. Comput. 10 (2) (2022) 1091–1104, http://dx.doi.org/10.1109/TETC.2021.3072421.

[11] Y. Wang, C. Wang, C. Gu, Y. Cui, M. O'Neill, W. Liu, A generic dynamic responding mechanism and secure authentication protocol for strong PUFs, IEEE Trans. Very Large Scale Integr. Syst. 30 (9) (2022) 1256–1268, http://dx.doi.org/10.1109/TVLSI.2022.3189953.

[12] L. Wu, Y. Hu, K. Zhang, W. Li, X. Xu, W. Chang, FLAM-PUF: a response-feedback-based lightweight anti-machine-learning-attack PUF, IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. 41 (11) (2022) 4433–4444, http://dx.doi.org/10.1109/TCAD.2022.3197696.

[13] Z. Huang, F. Zeng, Y. Chi, Y. Lin, Y. Lu, H. Liang, J. Bian, Y. Ouyang, T. Ni, X. Wen, BF PUF: A modeling attack-resistant strong PUF based on bent functions, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. 33 (8) (2025) 2299–2311, http://dx.doi.org/10.1109/TVLSI.2025.3569587.

[14] C.D. Cannière, B. Preneel, Trivium, in: M.J.B. Robshaw, O. Billet (Eds.), New Stream Cipher Designs - the ESTREAM Finalists, in: Lecture Notes in Computer Science, vol. 4986, Springer, 2008, pp. 244–266, http://dx.doi.org/10.1007/978-3-540-68351-3_18.

[15] F. Armknecht, V. Mikhalev, On lightweight stream ciphers with shorter internal states, in: G. Leander (Ed.), Fast Software Encryption - 22nd International Workshop, FSE 2015, Istanbul, Turkey, March 8-11, 2015, Revised Selected Papers, in: Lecture Notes in Computer Science, vol. 9054, Springer, 2015, pp. 451–470, http://dx.doi.org/10.1007/978-3-662-48116-5_22.

[16] M. Hamann, M. Krause, W. Meier, LIZARD - a lightweight stream cipher for power-constrained devices, IACR Trans. Symmetric Cryptol. 2017 (1) (2017) 45–79, http://dx.doi.org/10.13154/TOSC.V2017.I1.45-79.

[17] U. Rührmair, F. Sehnke, J. Sölter, G. Dror, S. Devadas, J. Schmidhuber, Modeling attacks on physical unclonable functions, 2010, p. 251, IACR Cryptol. ePrint Arch..

[18] B. Gassend, D.E. Clarke, M. van Dijk, S. Devadas, Silicon physical random functions, in: V. Atluri (Ed.), Proceedings of the 9th ACM Conference on Computer and Communications Security, CCS 2002, Washington, DC, USA, November 18-22, 2002, ACM, 2002, pp. 148–160, http://dx.doi.org/10.1145/586110.586132.

[19] U. Rührmair, J. Sölter, F. Sehnke, X. Xu, A. Mahmoud, V. Stoyanova, G. Dror, J. Schmidhuber, W.P. Burleson, S. Devadas, PUF modeling attacks on simulated and silicon data, 2013, p. 112, IACR Cryptol. ePrint Arch..

[20] N. Mukherjee, J. Rajski, G. Mrugalski, A. Pogiel, J. Tyszer, Ring generator: An ultimate linear feedback shift register, Computer 44 (6) (2011) 64–71, http://dx.doi.org/10.1109/MC.2010.334.

[21] J. Delvaux, R. Peeters, D. Gu, I. Verbauwhede, A survey on lightweight entity authentication with strong PUFs, ACM Comput. Surv. 48 (2) (2015) 26:1–26:42, http://dx.doi.org/10.1145/2818186.

[22] D. Hachenberger, D. Jungnickel, Shift register sequences, in: Topics in Galois Fields, Springer International Publishing, Cham, 2020, pp. 427–487, http://dx.doi.org/10.1007/978-3-030-60806-4_9.

[23] O.S. Rothaus, On "bent" functions, J. Comb. Theory A 20 (3) (1976) 300–305, http://dx.doi.org/10.1016/0097-3165(76)90024-8.

[24] J. Rajski, M. Trawka, J. Tyszer, B. Wlodarczak, H_2b: Crypto hash functions based on hybrid ring generators, IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. 43 (2) (2024) 442–455, http://dx.doi.org/10.1109/TCAD.2023.3320633.

[25] J. Seberry, X. Zhang, Highly nonlinear 0-1 balanced boolean functions satisfying strict avalanche criterion, in: J. Seberry, Y. Zheng (Eds.), Advances in Cryptology - AUSCRYPT '92, Workshop on the Theory and Application of Cryptographic Techniques, Gold Coast, Queensland, Australia, December 13-16, 1992, Proceedings, in: Lecture Notes in Computer Science, 718, Springer, 1992, pp. 145–155, http://dx.doi.org/10.1007/3-540-57220-1_58.

[26] P.H. Nguyen, D.P. Sahoo, R.S. Chakraborty, D. Mukhopadhyay, Security analysis of arbiter PUF and its lightweight compositions under predictability test, ACM Trans. Des. Autom. Electr. Syst. 22 (2) (2017) 20:1–20:28, http://dx.doi.org/10.1145/2940326.

[27] Y. Wei, T. Fox, V. Dumoulin, W. Rao, N. Devroye, APUF faults: Impact, testing, and diagnosis, in: C. Bolchini, I. Verbauwhede, E. Vatajelu (Eds.), 2022 Design, Automation & Test in Europe Conference & Exhibition, DATE 2022, Antwerp, Belgium, March 14-23, 2022, IEEE, 2022, pp. 442–447, http://dx.doi.org/10.23919/DATE54114.2022.9774575.

[28] N.N. Anandakumar, M.S. Hashmi, M.M. Tehranipoor, FPGA-based physical unclonable functions: A comprehensive overview of theory and architectures, Integr. 81 (2021) 175–194, http://dx.doi.org/10.1016/J.VLSI.2021.06.001.

[29] J. Liu, Y. Zhao, Y. Zhu, C. Chan, R.P. Martins, A weak PUF-assisted strong PUF with inherent immunity to modeling attacks and ultra-low BER, IEEE Trans. Circuits Syst. I Regul. Pap. 69 (12) (2022) 4898–4907, http://dx.doi.org/10.1109/TCSI.2022.3206214.

[30] H. He, P. Wang, X. Li, Y. Zhang, X. Zhang, Highly reliable RS-PUF based on reconfigurable gas sensor array, IEEE Sensors J. 24 (10) (2024) 16875–16882, http://dx.doi.org/10.1109/JSEN.2024.3375872.

[31] F. Pareschi, R. Rovatti, G. Setti, On statistical tests for randomness included in the NIST SP800-22 test suite and based on the binomial distribution, IEEE Trans. Inf. Forensics Secur. 7 (2) (2012) 491–505, http://dx.doi.org/10.1109/TIFS.2012.2185227.

[32] U. Rührmair, J. Sölter, F. Sehnke, X. Xu, A. Mahmoud, V. Stoyanova, G. Dror, J. Schmidhuber, W.P. Burleson, S. Devadas, PUF modeling attacks on simulated and silicon data, IEEE Trans. Inf. Forensics Secur. 8 (11) (2013) 1876–1891, http://dx.doi.org/10.1109/TIFS.2013.2279798.

[33] G.T. Becker, The gap between promise and reality: On the insecurity of XOR arbiter PUFs, in: T. Güneysu, H. Handschuh (Eds.), Cryptographic Hardware and Embedded Systems - CHES 2015 - 17th International Workshop, Saint-Malo, France, September 13-16, 2015, Proceedings, in: Lecture Notes in Computer Science, 9293, Springer, 2015, pp. 535–555, http://dx.doi.org/10.1007/978-3-662-48324-4_27.

[34] N. Hansen, The CMA evolution strategy: A tutorial, 2016, CoRR, abs/1604.00772 arXiv:1604.00772.

[35] C.M. Bishop, Pattern recognition and machine learning, 5th edition, in: Information science and statistics, Springer, 2007.

[36] K. Hornik, M.B. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, Neural Netw. 2 (5) (1989) 359–366, http://dx.doi.org/10.1016/0893-6080(89)90020-8.

[37] N. Wisiol, C. Gräbnitz, C. Mühl, B. Zengin, T. Soroceanu, N. Pirnay, K.T. Mursi, A. Baliuka, pypuf: Cryptanalysis of Physically Unclonable Functions, 2021, http://dx.doi.org/10.5281/zenodo.3901410.

[38] H. Awano, T. Sato, Ising-PUF: A machine learning attack resistant PUF featuring lattice like arrangement of arbiter-PUFs, in: J. Madsen, A.K. Coskun (Eds.), 2018 Design, Automation & Test in Europe Conference & Exhibition, DATE 2018, Dresden, Germany, March 19-23, 2018, IEEE, 2018, pp. 1447–1452, http://dx.doi.org/10.23919/DATE.2018.8342239.

[39] N. Wisiol, C. Mühl, N. Pirnay, P.H. Nguyen, M. Margraf, J. Seifert, M. van Dijk, U. Rührmair, Splitting the interpose PUF: a novel modeling attack strategy, IACR Trans. Cryptogr. Hardw. Embed. Syst. 2020 (3) (2020) 97–120, http://dx.doi.org/10.13154/TCHES.V2020.I3.97-120.

[40] J. Zhang, C. Shen, Z. Guo, Q. Wu, W. Chang, CT PUF: configurable tristate PUF against machine learning attacks for IoT security, IEEE Internet Things J. 9 (16) (2022) 14452–14462, http://dx.doi.org/10.1109/JIOT.2021.3090475.